

GUJARAT TECHNOLOGICAL UNIVERSITY

Navyug Vidyabhavan Trust

C. K. Pithawalla College of Engineering and Technology, Surat

A

Project Report

On

(Healthcare Analytics)

Prepared as a part of the requirements for the subject of

Python for Data Science (BE05000231)

B. E. III, Semester – V

(Computer Engineering)

Submitted by:

Sr. No.	Enrolment No	Name of Student	Division
1	240090107070	Om Koradia	C
2	240090107087	Meetrajsinh Vala	C
3	240090107097	Jeel Moradiya	C
4	240090107175	Daksh Rana	C

Academic year (2026-27)

Table of Contents

Sr. No	Title	Page No
1	Introduction	2
2	Problem Statement	2
3	Objectives	3
4	Dataset Description	3
5	Tools and Technologies Used	4
6	Project Methodology	5
7	Data Loading	6
8	Understanding the Dataset	7
9	Data Cleaning	8
10	Patient Age Analysis	9
11	Gender Analysis	10
12	Department Analysis	11
13	Treatment Analysis	12
14	Visit Type Analysis	13
15	Treatment Cost Analysis	14
16	Length of Hospital Stay	14
17	Data Visualization	15
18	Statistics Analysis	19
19	Numpy	20
20	Relationship Between Stay and Cost	22
21	Machine Learning	26
22	Implementation	33
23	Research/Idea Development	34
24	Team Contribution	34
25	Results	35
26	Limitations	35
27	Future Scope	36
28	Conclusion	36
29	References	37

1. Introduction

Healthcare data contains useful information about patients, treatments, hospital departments and treatment costs.

In this project, we use **Python** to analyze healthcare data and find useful information from it.

We analyze things like:

- Patient age
- Gender
- Region
- Hospital department
- Treatment type
- Visit type
- Length of hospital stay
- Treatment cost

Our dataset contains **5,000 patient records and 10 columns**.

The main purpose of this project is to understand how Python can be used to convert raw healthcare data into useful information.

Link - [healthcare_patient_analytics.csv](#)

2. Problem Statement

Hospitals have a large amount of patient data. Looking at raw data manually is difficult.

Therefore, we want to develop a simple **Healthcare Analytics system using Python** that can:

- Analyze patient data
- Find common treatments
- Compare hospital departments
- Analyze emergency and routine visits
- Study treatment costs
- Study hospital stay duration
- Display the results using graphs
- Use machine learning for prediction

3. Objectives

The main objectives of our project are:

- To understand the healthcare dataset.
- To clean and prepare the data.
- To analyze patient information.
- To find the most common hospital department.
- To analyze different treatment types.
- To compare emergency and routine visits.
- To calculate basic statistics.
- To create graphs for easy understanding.
- To apply a simple machine learning model.
- To understand how data science can be used in healthcare.

4. Dataset Description

We used a healthcare patient dataset containing **5,000 records**.

The dataset has the following columns:

Column	Simple Meaning
patient_id	Unique patient number
visit_date	Date of visit
age_group	Age category
gender	Male or Female
region	Patient's region
department	Hospital department
treatment_type	Type of treatment
visit_type	Emergency or Routine
length_of_stay_days	Number of days in hospital
treatment_cost	Cost of treatment

For example, the dataset contains departments such as **General Medicine, Orthopedics, Pediatrics, Neurology and Cardiology**. It also contains treatment types such as **Medication, Surgery, Therapy and Observation**.

5. Tools and Technologies Used

We used:

Programming Language

Python

Software

- Jupyter Notebook / Google Colab

Libraries

Pandas – used for handling data.

NumPy – used for numerical calculations.

Matplotlib – used for creating graphs.

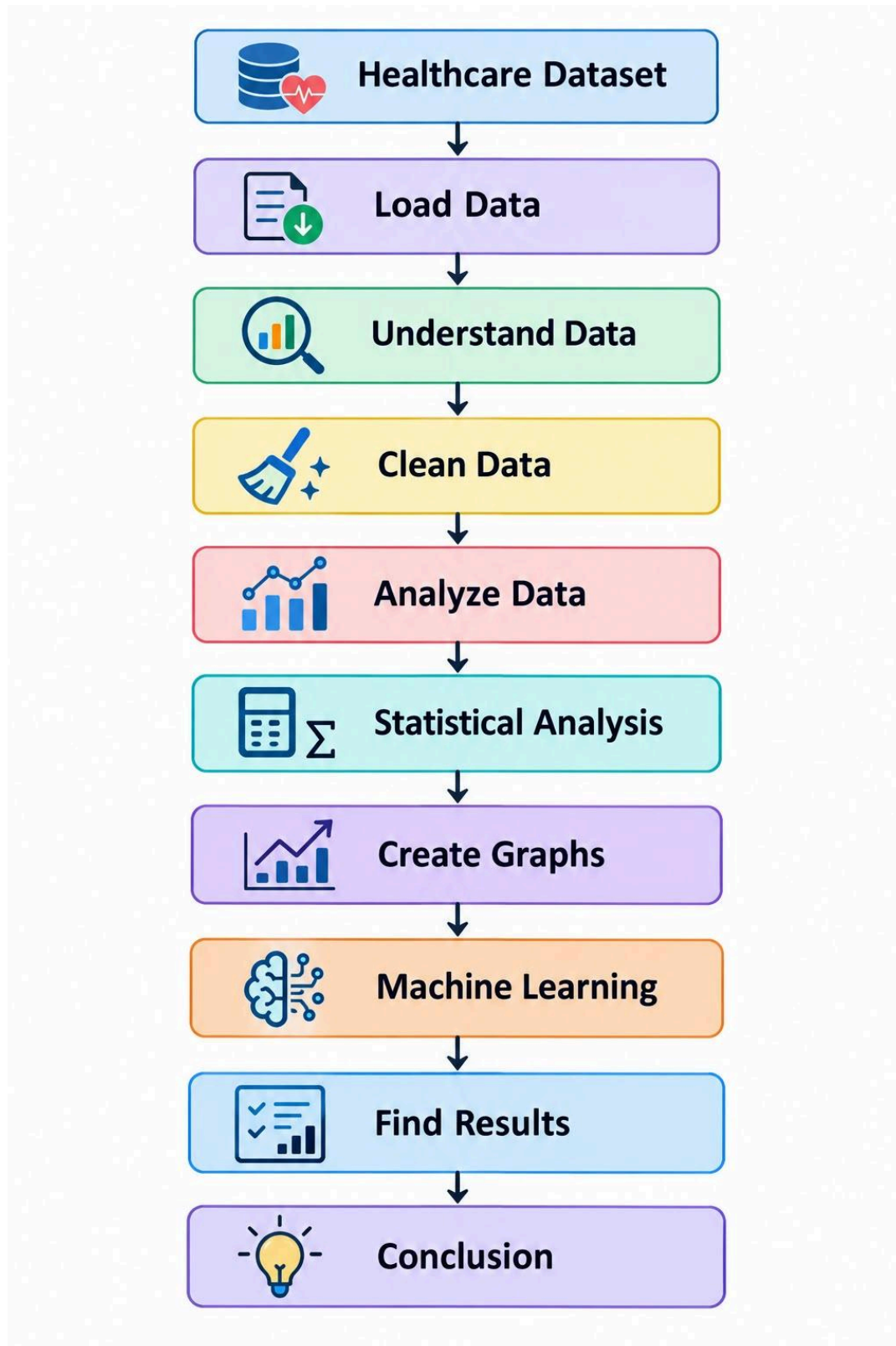
Seaborn – used for better data visualization.

SciPy – used for statistical analysis.

Scikit-learn – used for machine learning.

6. Project Methodology

The project follows these simple steps:



7. Data Loading

First, we load our dataset into Python using Pandas.

Code:

```
import pandas as pd

df = pd.read_csv("healthcare_data.csv")

print(df.head())
```

Output:

	patient_id	visit_date	age_group	gender	region	department	\
0	1	2022-01-01 00:00:00	31-45	Male	West	General Medicine	
1	2	2022-01-01 01:00:00	60+	Female	West	Orthopedics	
2	3	2022-01-01 02:00:00	46-60	Male	South	Pediatrics	
3	4	2022-01-01 03:00:00	31-45	Female	North	Neurology	
4	5	2022-01-01 04:00:00	18-30	Female	North	Neurology	
	treatment_type	visit_type	length_of_stay_days	treatment_cost			
0	Medication	Emergency	5.8	59151.0			
1	Surgery	Routine	5.1	30272.0			
2	Observation	Routine	7.9	67498.0			
3	Medication	Routine	5.0	29896.0			
4	Therapy	Routine	0.0	36208.0			

Explanation

`pandas` helps us work with the dataset.

`read_csv()` reads the CSV file.

`head()` displays the first few rows.

8. Understanding the Dataset

We use the following commands:

Code:

```
print(df.shape)
print(df.info())
print(df.describe())
```

Output:

```
(5000, 10)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   patient_id            5000 non-null   int64  
1   visit_date            5000 non-null   object  
2   age_group             5000 non-null   object  
3   gender                5000 non-null   object  
4   region               5000 non-null   object  
5   department            5000 non-null   object  
6   treatment_type        5000 non-null   object  
7   visit_type            5000 non-null   object  
8   length_of_stay_days   5000 non-null   float64 
9   treatment_cost        5000 non-null   float64 
dtypes: float64(2), int64(1), object(7)
memory usage: 390.8+ KB
None
```

	patient_id	length_of_stay_days	treatment_cost
count	5000.000000	5000.000000	5000.000000
mean	2500.500000	4.059660	54915.471800
std	1443.520003	1.928847	19481.160487
min	1.000000	0.000000	746.000000
25%	1250.750000	2.700000	41244.750000
50%	2500.500000	4.000000	55123.500000
75%	3750.250000	5.400000	68012.000000
max	5000.000000	11.900000	119307.000000

Explanation

- **shape** tells us the number of rows and columns.
- **info()** tells us about the columns and their data types.
- **describe()** gives basic statistical information.

Our dataset contains **5,000 patient records and 10 columns**.

9. Data Cleaning

Before analysis, we check whether the dataset contains missing or duplicate values.

Code:

```
print(df.isnull().sum())  
print(df.duplicated().sum())
```

Output:

```
patient_id      0  
visit_date      0  
age_group       0  
gender          0  
region          0  
department      0  
treatment_type  0  
visit_type      0  
length_of_stay_days  0  
treatment_cost  0  
dtype: int64  
0
```

Explanation

- `isnull()` checks missing values.
- `duplicated()` checks repeated records.

Data cleaning is important because incorrect or missing data can affect our results.

10. Patient Age Analysis

We analyze different age groups.

The dataset contains:

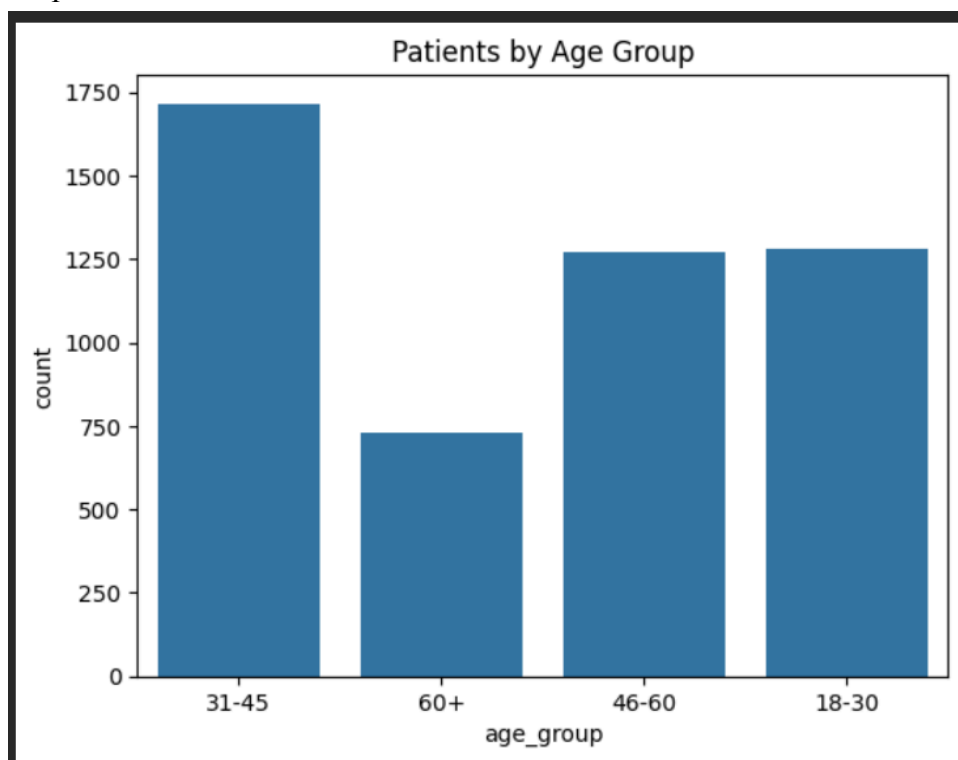
- 18–30
- 31–45
- 46–60
- 60+

We can create a graph using:

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.countplot(data=df, x="age_group")
plt.title("Patients by Age Group")
plt.show()
```

Output:



Finding

The **31–45** age group has the highest number of patients in the dataset.

11. Gender Analysis

We compare male and female patients.

Code:

```
print(df["gender"].value_counts())
```

Output:

```
gender
Male      2508
Female    2492
Name: count, dtype: int64
```

Result

Gender	Patients
Male	2,508
Female	2,492

Finding

The number of male and female patients is almost equal.

12. Department Analysis

The dataset contains different hospital departments.

We can analyze them using:

Code:

```
print(df["department"].value_counts())
```

Output:

```
department
Orthopedics      1058
Cardiology        995
General Medicine  991
Pediatrics        989
Neurology         967
Name: count, dtype: int64
```

The departments include:

- Orthopedics
- Cardiology
- General Medicine
- Pediatrics
- Neurology

Finding

- **Orthopedics has the highest number of patient records.**

13. Treatment Analysis

The dataset contains four main treatment types:

- Observation
- Surgery
- Medication
- Therapy

We use:

Code:

```
print(df["treatment_type"].value_counts())
```

Output:

```
treatment_type
Observation    1270
Surgery        1262
Medication     1236
Therapy        1232
Name: count, dtype: int64
```

Finding

- **Observation is the most common treatment type** in the dataset.

14. Visit Type Analysis

There are two types of visits:

- Emergency
- Routine

We use:

Code:

```
print(df["visit_type"].value_counts())
```

Output:

```
visit_type
Routine      3432
Emergency     1568
Name: count, dtype: int64
```

Result

Visit Type	Patients
Routine	3,432
Emergency	1,568

Finding

Routine visits are more common than emergency visits.

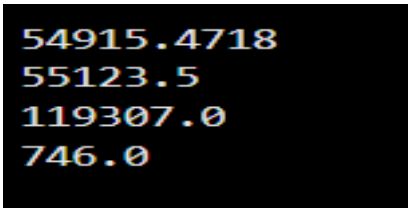
15. Treatment Cost Analysis

We analyze the treatment cost using Pandas.

Code:

```
print(df["treatment_cost"].mean())  
print(df["treatment_cost"].median())  
print(df["treatment_cost"].max())  
print(df["treatment_cost"].min())
```

Output:



```
54915.4718  
55123.5  
119307.0  
746.0
```

Results

Measurement	Value
Average Cost	₹54,915.47
Median Cost	₹55,123.50
Minimum Cost	₹746
Maximum Cost	₹119,307

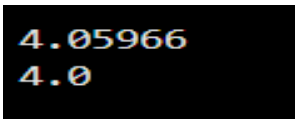
16. Length of Hospital Stay

We also analyze how long patients stay in the hospital.

Code:

```
print(df["length_of_stay_days"].mean())  
print(df["length_of_stay_days"].median())
```

Output:



```
4.05966  
4.0
```

Result

The average hospital stay is approximately **4 days**.

17. Data Visualization

Data visualization helps us understand the healthcare data easily through graphs and charts.

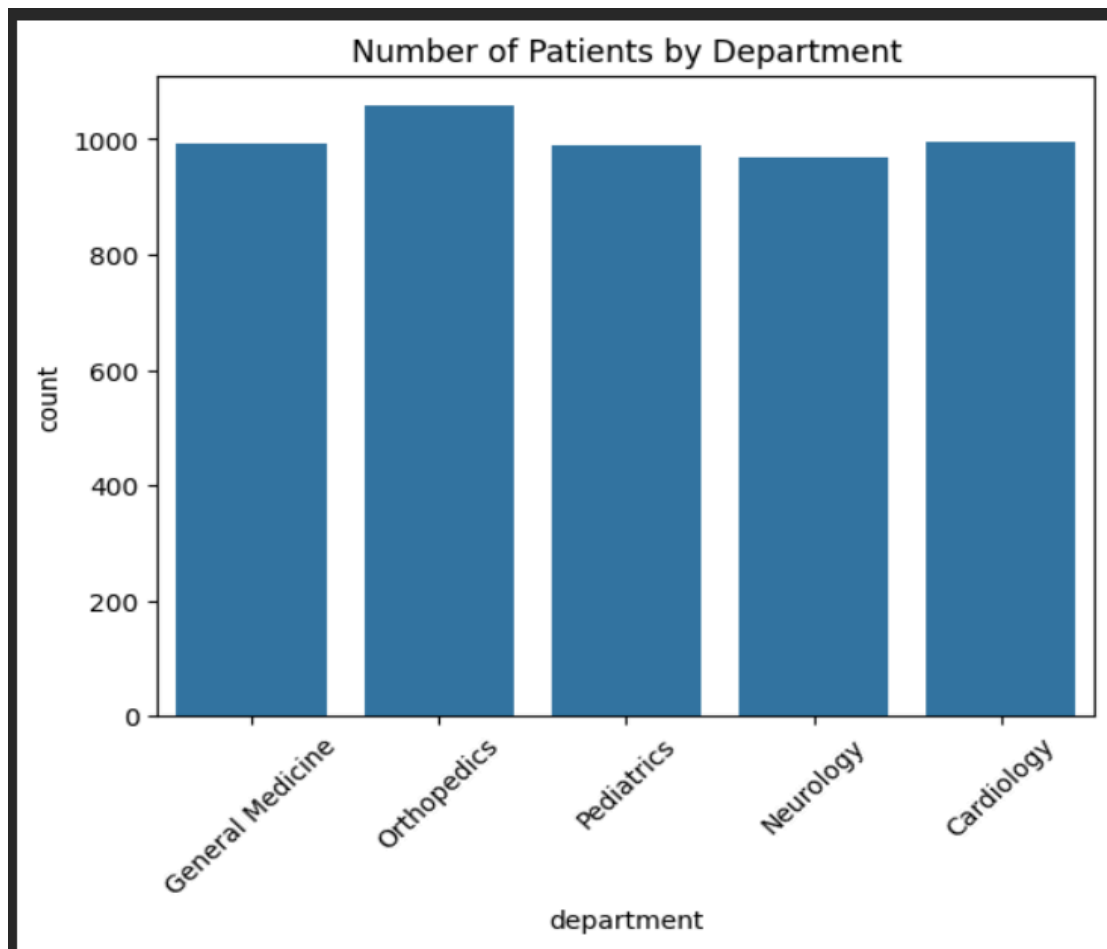
In our project, we used different types of graphs according to the type of data.

17.1 Bar Chart – Patients by Department

Code:

```
sns.countplot(data=df, x="department")  
plt.title("Number of Patients by Department")  
plt.xticks(rotation=45)  
plt.show()
```

Output:



Explanation:

This graph compares the number of patients in different hospital departments.

Finding:

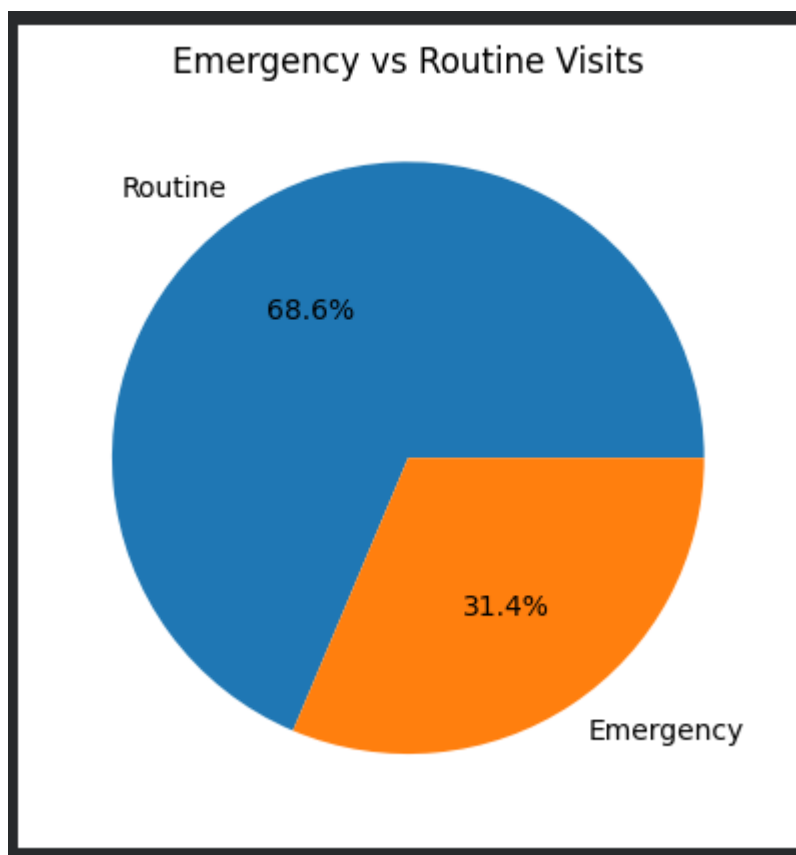
Orthopedics has the highest number of patient records.

17.2 Pie Chart – Emergency vs Routine Visits

Code:

```
df["visit_type"].value_counts().plot(  
    kind="pie",  
    autopct="%1.1f%%"  
)  
plt.title("Emergency vs Routine Visits")  
plt.ylabel("")  
plt.show()
```

Output:



Explanation:

This pie chart shows the percentage of emergency and routine visits.

Finding:

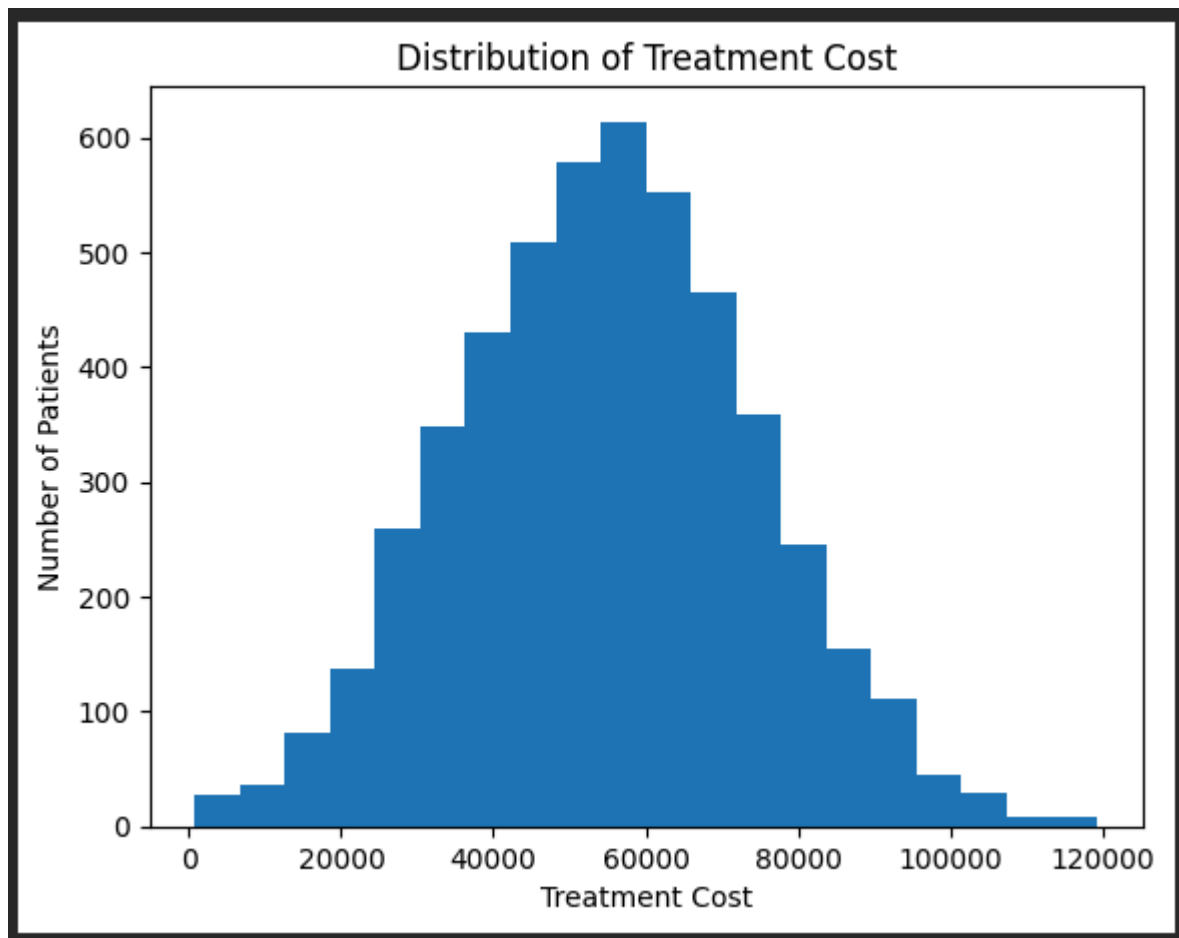
Routine visits are more common than emergency visits.

17.3 Histogram – Treatment Cost Distribution

Code:

```
plt.hist(df["treatment_cost"], bins=20)
plt.title("Distribution of Treatment Cost")
plt.xlabel("Treatment Cost")
plt.ylabel("Number of Patients")
plt.show()
```

Output:



Explanation:

The histogram shows how treatment costs are distributed among patients.

Finding:

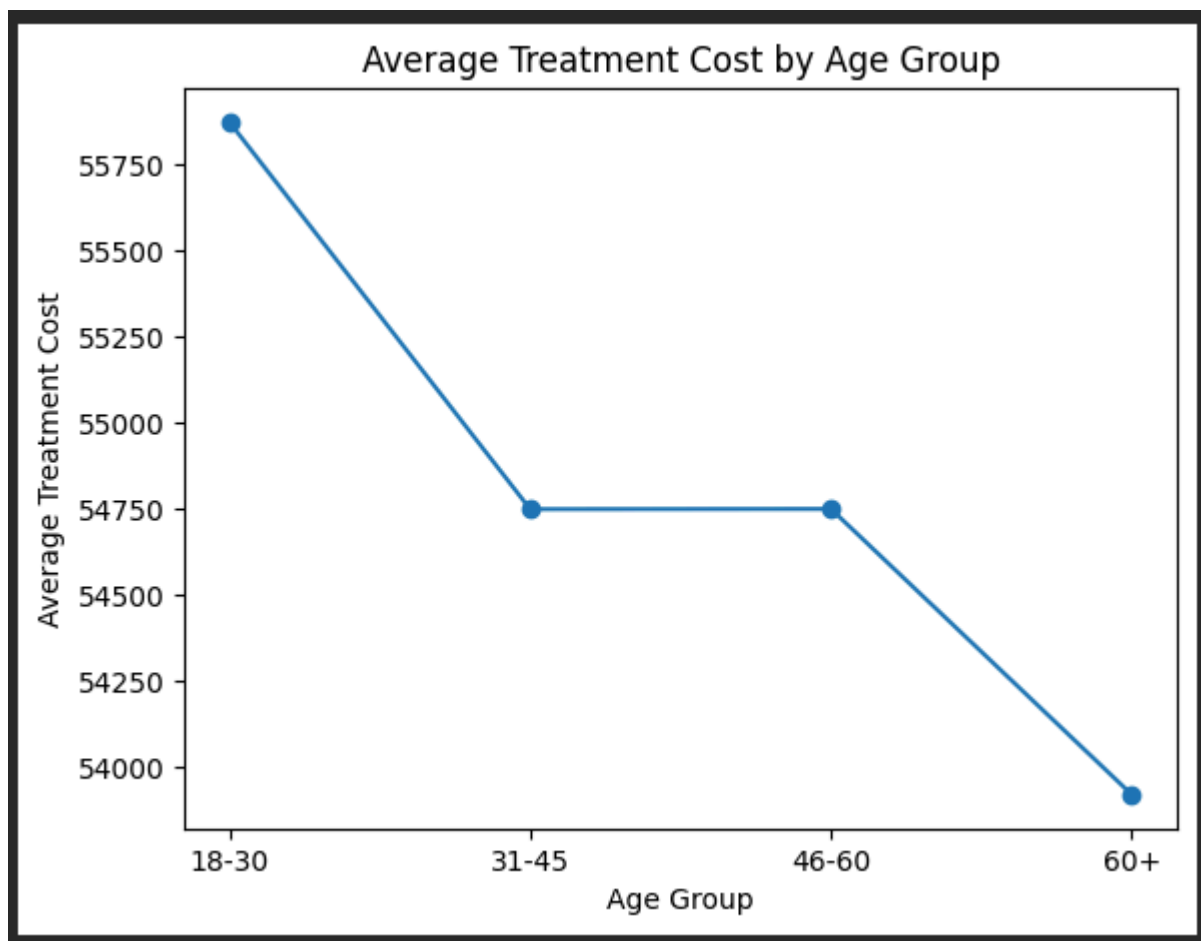
It helps us understand which treatment-cost ranges occur more frequently.

17.4 Line Graph – Average Treatment Cost by Age Group

Code:

```
avg_cost = df.groupby("age_group")["treatment_cost"].mean()
plt.plot(avg_cost.index, avg_cost.values, marker="o")
plt.title("Average Treatment Cost by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Average Treatment Cost")
plt.show()
```

Output:



Explanation:

This graph compares the average treatment cost for different age groups.

Finding:

It helps us compare how the average treatment cost changes between age groups.

18. Statistical Analysis

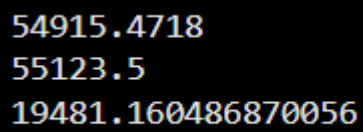
We use statistics to understand the numerical data.

For example:

Code:

```
print(df["treatment_cost"].mean())  
print(df["treatment_cost"].median())  
print(df["treatment_cost"].std())
```

Output:



```
54915.4718  
55123.5  
19481.160486870056
```

We calculate:

- Mean
- Median
- Standard deviation

19. Numpy

Average, Minimum and Maximum

We use NumPy for numerical calculations and working with numerical data.

For example:

Code:

```
import numpy as np
```

```
cost = df["treatment_cost"]
```

```
print("Average Cost:", np.mean(cost))
```

```
print("Minimum Cost:", np.min(cost))
```

```
print("Maximum Cost:", np.max(cost))
```

Output:

```
Average Cost: 54915.4718  
Minimum Cost: 746.0  
Maximum Cost: 119307.0
```

We calculate:

1. Mean – average treatment cost
2. Minimum – lowest treatment cost
3. Maximum – highest treatment cost

Sum of Treatment Costs

We use **NumPy** to perform numerical calculations on the treatment cost data.

For example:

Code:

```
import numpy as np

cost = df["treatment_cost"]

print("Total Treatment Cost:", np.sum(cost))
```

Output:



```
Total Treatment Cost: 274577359.0
```

We calculate:

- **Sum** – total treatment cost of all patients

20. Relationship Between Stay and Cost

Hospital stay duration and treatment cost are two important factors in healthcare data. In this analysis, we study whether the length of a patient's hospital stay is related to the treatment cost.

We use a **scatter plot** to visualize the relationship between these two variables. The **X-axis** represents the length of hospital stay in days, while the **Y-axis** represents the treatment cost in rupees.

20.1 Seaborn Scatter Plot

We use the **Seaborn** library to create a scatter plot.

The graph also uses different colors for **Emergency** and **Routine** visits, which helps us compare the two types of visits.

Explanation

- The X-axis shows the **length of hospital stay**.
- The Y-axis shows the **treatment cost**.
- Each point represents a patient record.
- Different colors represent different **visit types**.
- The graph helps us observe how treatment cost varies with hospital stay.

We can check whether hospital stay and treatment cost are related.

Code:

```
print(  
    df[["length_of_stay_days", "treatment_cost"]].corr()  
)
```

Output:

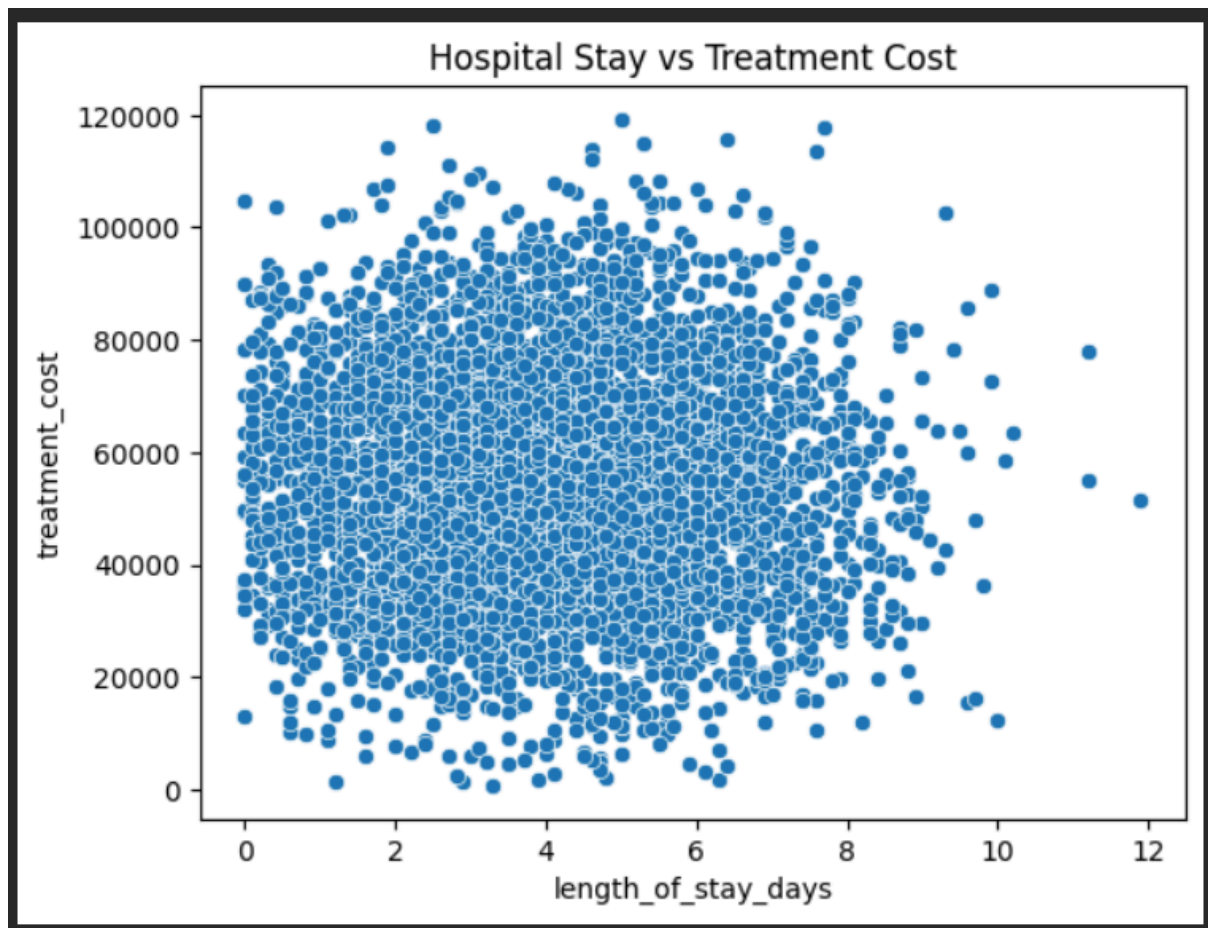
```
length_of_stay_days  length_of_stay_days  treatment_cost  
length_of_stay_days      1.000000      -0.022281  
treatment_cost          -0.022281      1.000000
```

We can also visualize it:

Code:

```
sns.scatterplot(  
    data=df,  
    x="length_of_stay_days",  
    y="treatment_cost"  
)  
plt.title("Hospital Stay vs Treatment Cost")  
plt.show()
```

Output:



20.2 Interactive Visualization

To make the graph easier to explore, we also created an **interactive scatter plot using Plotly**.

The interactive graph provides features such as:

- Zoom in
- Zoom out
- Pan or move the graph
- Reset the graph
- View individual data points
- Download the graph

This allows us to examine specific areas of the healthcare data more closely.

Why Interactive Visualization?

A normal graph provides a fixed view of the data. An interactive graph allows the user to explore the data by zooming and moving around the graph.

This is useful when there are many patient records because individual data points can be difficult to observe in a normal static graph.

Code:

```
import plotly.express as px
```

```
fig = px.scatter(
    df,
    x="length_of_stay_days",
    y="treatment_cost",
    color="visit_type",
    title="Hospital Stay vs Treatment Cost",
    labels={
        "length_of_stay_days": "Length of Stay (Days)",
        "treatment_cost": "Treatment Cost (₹)",
        "visit_type": "Visit Type"
    }
)
```

)

```
fig.update_layout(
```

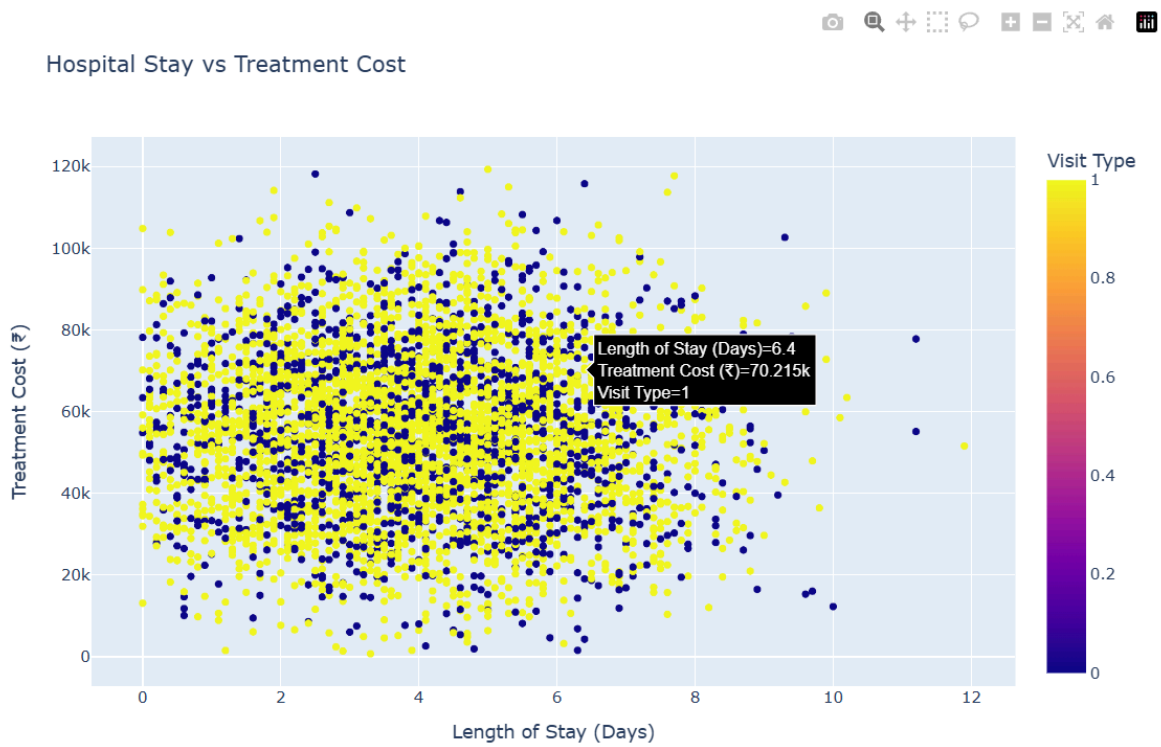
```
    width=900,
```

```
    height=600
```

```
)
```

```
fig.show()
```

Output:



21. Machine Learning

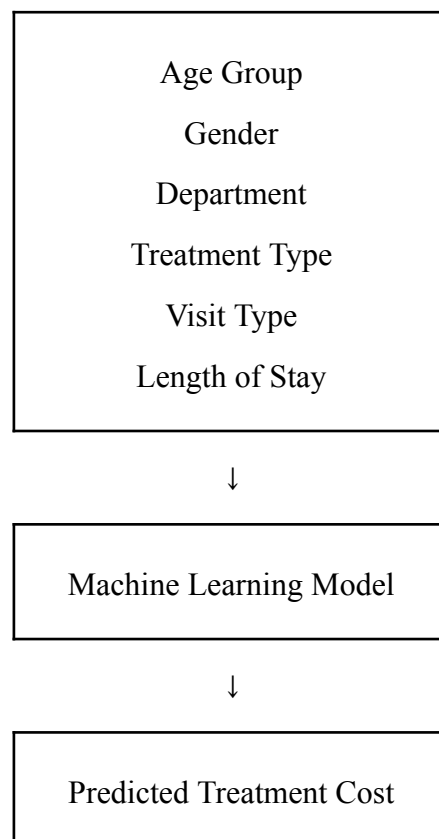
As part of the project, we also apply a simple machine learning model.

Our prediction problem

We try to predict treatment cost using other patient information, such as: Age Group, Gender, Department, Treatment Type, Visit Type, and Length of Stay.

These features are passed into a Machine Learning Model, which outputs the Predicted Treatment Cost.

We use Random Forest Regression For example:



We use **Random Forest Regression**.

Why Random Forest?

Random Forest is a machine learning algorithm that combines multiple decision trees to make a prediction.

Code:

```

import pandas as pd

from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor

df = pd.read_csv("healthcare_patient_analytics.csv")

encoders = {}

columns = [
    "age_group",
    "gender",
    "department",
    "treatment_type",
    "visit_type"
]

for col in columns:
    encoders[col] = LabelEncoder()
    df[col] = encoders[col].fit_transform(df[col])

X_stay = df[
    "age_group",
    "gender",
    "department",
    "treatment_type",
    "visit_type"
]

y_stay = df["length_of_stay_days"]

```

```
stay_model = RandomForestRegressor(  
    n_estimators=100,  
    random_state=42  
)
```

```
stay_model.fit(X_stay, y_stay)
```

```
X_cost = df[  
    [  
        "age_group",  
        "gender",  
        "department",  
        "treatment_type",  
        "visit_type",  
        "length_of_stay_days"  
    ]  
]
```

```
y_cost = df["treatment_cost"]
```

```
cost_model = RandomForestRegressor(  
    n_estimators=100,  
    random_state=42  
)
```

```
cost_model.fit(X_cost, y_cost)
```

```
print("\n=== PATIENT DETAILS ===\n")
```

```
age = input(  
    "Enter Age Group (18-30, 31-45, 46-60, 60+): "  
)
```

```
gender = input(  
    "Enter Gender (Male/Female): "  
)
```

```
treatment = input(  
    "Enter Treatment Type: "  
)
```

```
department = input(  
    "Enter Department: "  
)
```

```
visit = input(  
    "Enter Visit Type (Emergency/Routine): "  
)
```

```
patient = pd.DataFrame({  
    "age_group": [  
        encoders["age_group"].transform([age])[0]  
    ],  
    "gender": [  
        encoders["gender"].transform([gender])[0]  
    ],  
    "department": [  
        encoders["department"].transform([department])[0]  
    ]  
})
```

```

],

    "treatment_type": [
        encoders["treatment_type"].transform([treatment])[0]
    ],

    "visit_type": [
        encoders["visit_type"].transform([visit])[0]
    ]
})

```

```

predicted_stay = stay_model.predict(patient)[0]

```

```

# Create minimum and maximum stay

```

```

min_stay = max(1, round(predicted_stay - 2))

```

```

max_stay = max(min_stay, round(predicted_stay + 2))

```

```

# Predict cost using predicted average stay

```

```

patient_base = patient.copy()

```

```

patient_base["length_of_stay_days"] = round(predicted_stay)

```

```

base_cost = cost_model.predict(patient_base)[0]

```

```

# Calculate average cost per hospital day

```

```

cost_per_day = df["treatment_cost"].sum() / df["length_of_stay_days"].sum()

```

```

# Keep base cost related to patient condition

```

```

# and adjust it according to stay duration.

```

```

min_cost = base_cost - (
    (predicted_stay - min_stay) * cost_per_day
)

max_cost = base_cost + (
    (max_stay - predicted_stay) * cost_per_day
)

# Make sure maximum cost is always >= minimum cost
if max_cost < min_cost:
    max_cost = min_cost

print("-- HEALTHCARE PREDICTION --")

print("\nPatient Details")
print("-----")

print("Age Group      :", age)
print("Gender         :", gender)
print("Treatment       :", treatment)
print("Department      :", department)
print("Visit Type      :", visit)

print("\n-- PREDICTION --")

print("\nMinimum Prediction")
print("Length of Stay :", min_stay, "days")
print("Treatment Cost : ₹", round(min_cost, 2))

print("\nMaximum Prediction")

```

```
print("Length of Stay :", max_stay, "days")
print("Treatment Cost : ₹", round(max_cost, 2))
```

Output:

```
=== PATIENT DETAILS ===

Enter Age Group (18-30, 31-45, 46-60, 60+): 46-60
Enter Gender (Male/Female): Male
Enter Treatment Type: Surgery
Enter Department: Cardiology
Enter Visit Type (Emergency/Routine): Emergency
-- HEALTHCARE PREDICTION --

Patient Details
-----
Age Group      : 46-60
Gender         : Male
Treatment      : Surgery
Department     : Cardiology
Visit Type     : Emergency

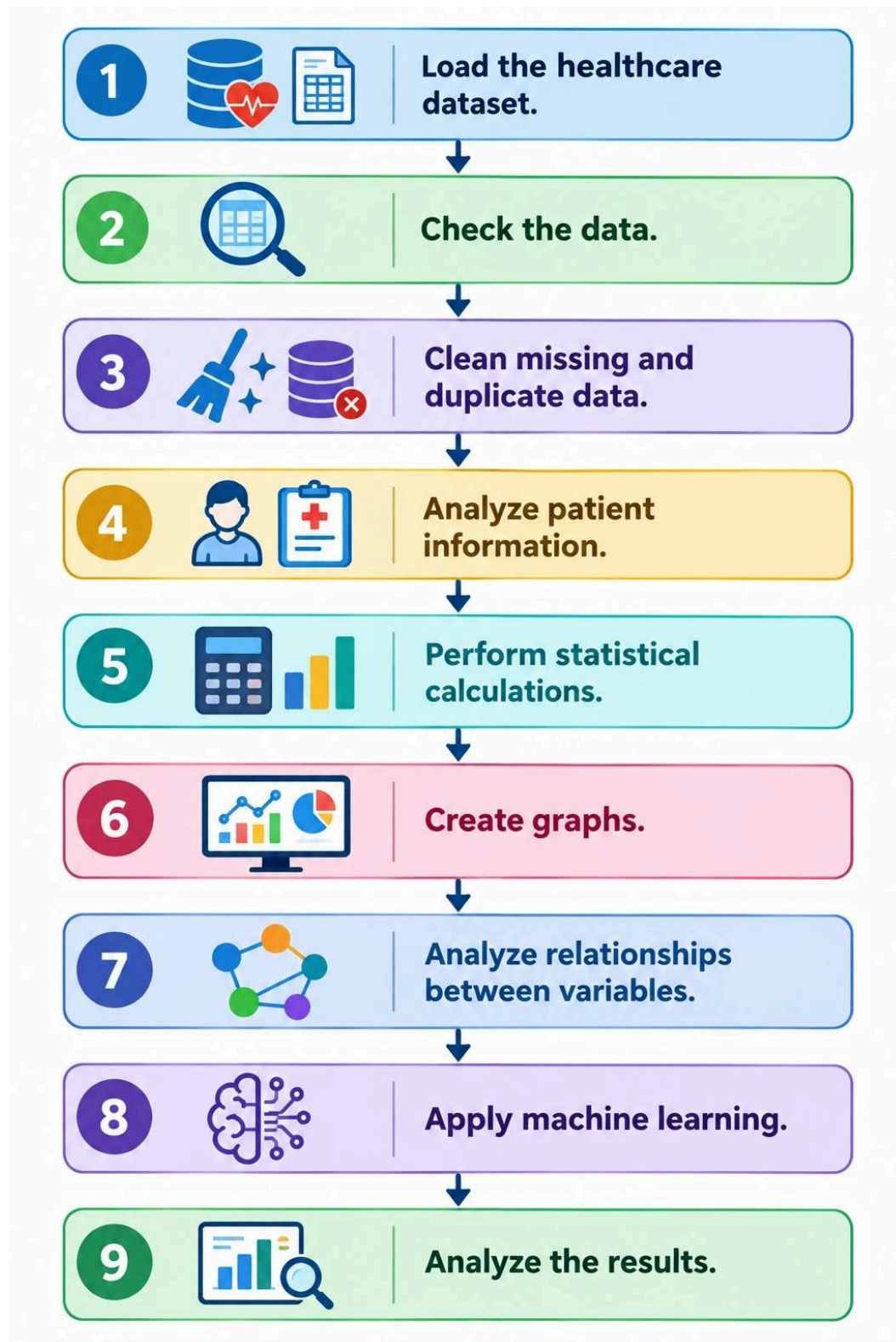
-- PREDICTION --

Minimum Prediction
Length of Stay : 2 days
Treatment Cost : ₹ 14263.62

Maximum Prediction
Length of Stay : 6 days
Treatment Cost : ₹ 68372.06
```

22. Implementation

Our implementation includes:



23. Research / Idea Development

Healthcare is an important real-world application of data science.

Hospitals have large amounts of patient information. We selected Healthcare Analytics because Python can help us analyze this information easily.

Our idea was to take patient data and answer simple questions such as:

- Which department has more patients?
- Which treatment is more common?
- Are emergency visits less common than routine visits?
- What is the average treatment cost?
- How long do patients usually stay?
- Can treatment cost be predicted?

This helped us connect the concepts learned in Python for Data Science with a real-world application.

24. Team Contribution

Team Member	Work
Member 1	Research and dataset
Member 2	Data cleaning
Member 3	Data analysis, statistics and visualization
Member 4	Machine learning, results and documentation

Team Collaboration.

All four members worked together on the project. The work was divided among the members according to different tasks. Each member completed their assigned work and participated in discussing the results, solving problems and preparing the final report and presentation.

25. Results

From our analysis, we found:

1. The **31–45 age group** has the highest number of patients.
2. Male and female patients are almost equally represented.
3. **Orthopedics** has the highest number of patient records.
4. **Observation** is the most common treatment type.
5. **Routine visits** are more common than emergency visits.
6. The average treatment cost is approximately **₹54,915**.
7. The average hospital stay is approximately **4 days**.
8. Graphs make the healthcare data easier to understand.
9. Machine learning can be used to predict treatment cost.

Google Colab Link - [PDS CIPAT](#)

26. Limitations

Our project has some limitations:

- The dataset contains limited patient information.
- It does not contain detailed medical information such as specific diseases or symptoms.
- Machine learning predictions depend on the quality of the available data.
- The results are based only on the given dataset.
- The dataset does not include information about previous medical history or medications.
- The machine learning model uses a limited number of features, which may affect the accuracy of predictions.

27. Future Scope

In the future, the project can be improved by adding:

- More patient records
- Disease information
- Medical test results
- Doctor information
- Hospital capacity
- Real-time healthcare data
- More advanced machine learning models
- A web-based healthcare analytics dashboard

28. Conclusion

Healthcare Analytics shows how Python can be used to understand healthcare data.

In our project, we used **Pandas, NumPy, Matplotlib, Seaborn, SciPy and Scikit-learn** for data analysis, statistics, visualization and machine learning.

We analyzed patient demographics, departments, treatments, visits, hospital stay and treatment costs.

The project helped us understand how the concepts of **Python for Data Science can be applied to a real-world healthcare problem.**

29. References

1. Python Documentation
<https://docs.python.org/3>
2. Pandas Documentation
<https://pandas.pydata.org/docs>
3. NumPy Documentation
<https://numpy.org/doc>
4. SciPy Documentation
<https://docs.scipy.org/doc/scipy>
5. Matplotlib Documentation
<https://matplotlib.org/stable>
6. Seaborn Documentation
<https://seaborn.pydata.org>
7. Scikit-learn Documentation
<https://scikit-learn.org>
8. Plotly Python Documentation
<https://plotly.com/python>
9. Google Colab

10. GitHub – Project Repository
<https://github.com/jeelmoradiya078-lgtm/CIPAT-Healthcare-Analytics.git>